

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – PENCARIAN NILAI MIN/MAX



NAMA : Raka Yulia Vegora

NIM : 109082500201

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pencarian Nilai Minimum dan Maksimum

Pencarian nilai minimum dan maksimum merupakan salah satu proses dasar dalam algoritma pemrograman. Proses ini digunakan untuk menemukan nilai terkecil dan nilai terbesar dari sekumpulan data. Data tersebut biasanya disimpan dalam bentuk array, list, atau struktur data lainnya.

Nilai **minimum** adalah nilai paling kecil dari seluruh elemen data, sedangkan nilai **maksimum** adalah nilai paling besar dari seluruh elemen data. Pencarian nilai min/max sering digunakan dalam berbagai kasus, seperti mencari nilai tertinggi dan terendah mahasiswa, menentukan harga termurah dan termahal, mencari suhu paling rendah dan paling tinggi, serta menganalisis data numerik lainnya.

Algoritma Pencarian Nilai Minimum

Langkah-langkah pencarian nilai minimum adalah sebagai berikut:

1. Tentukan elemen pertama sebagai nilai minimum sementara.
2. Bandingkan nilai minimum sementara dengan elemen berikutnya.
3. Jika elemen berikutnya lebih kecil, maka elemen tersebut menjadi nilai minimum baru.
4. Ulangi proses sampai seluruh elemen diperiksa.
5. Nilai minimum akhir adalah nilai terkecil dari seluruh data.

Algoritma Pencarian Nilai Maksimum

Langkah-langkah pencarian nilai maksimum adalah sebagai berikut:

1. Tentukan elemen pertama sebagai nilai maksimum sementara.
2. Bandingkan nilai maksimum sementara dengan elemen berikutnya.
3. Jika elemen berikutnya lebih besar, maka elemen tersebut menjadi nilai maksimum baru.
4. Ulangi proses sampai seluruh elemen diperiksa.
5. Nilai maksimum akhir adalah nilai terbesar dari seluruh data.

Kompleksitas Algoritma

Algoritma pencarian nilai minimum dan maksimum memiliki kompleksitas waktu $O(n)$, karena setiap elemen dalam data harus diperiksa satu per satu. Jika jumlah data semakin banyak, maka jumlah proses perbandingan juga akan bertambah secara linear.

Kompleksitas ruang dari algoritma ini adalah $O(1)$, karena hanya membutuhkan beberapa variabel tambahan seperti min, max, dan indeks perulangan. Algoritma ini tidak memerlukan penyimpanan tambahan yang besar.

B. Guided

1. minmaxarray.go

```
1 package main
2
3 import "fmt"
4
5 func nilaiTerbesar(array []int) (int, int) {
6     var idx int
7     var nilaiTerbesar int = array[0]
8     var idxDitemukan int = 0
9     for idx = 1; idx < len(array); idx++ {
10         if array[idx] > nilaiTerbesar {
11             nilaiTerbesar = array[idx]
12             idxDitemukan = idx
13         }
14     }
15     return nilaiTerbesar, idxDitemukan
16 }
17
18 func nilaiTerkecil(array []int) (int, int) {
19     var idx int
20     var nilaiTerkecil int = array[0]
21     var idxDitemukan int = 0
22     for idx = 1; idx < len(array); idx++ {
23         if array[idx] < nilaiTerkecil {
24             nilaiTerkecil = array[idx]
25             idxDitemukan = idx
26         }
27     }
28     return nilaiTerkecil, idxDitemukan
29 }
30
31 func main() {
32     var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}
33
34     var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
35     var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])
36
37     fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d", nilaiMin, idxNilaiMin)
38     fmt.Println()
39     fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d", nilaiMaks, idxNilaiMaks)
40
41 }
```

Penjelasan :

Program tersebut digunakan untuk mencari nilai terbesar dan nilai terkecil dari sebuah array. Pertama, program membuat fungsi nilaiTerbesar yang menerima array bertipe integer, kemudian membandingkan setiap elemen array untuk menemukan nilai paling

besar beserta indeks tempat nilai tersebut berada. Selanjutnya, program juga membuat fungsi nilaiTerkecil yang bekerja dengan cara yang hampir sama, tetapi digunakan untuk mencari nilai paling kecil beserta indeksinya. Pada fungsi main, dibuat sebuah array bernama arrAngka yang berisi lima nilai, yaitu 45, 23, 98, 66, dan 12. Setelah itu, program memanggil fungsi nilaiTerbesar dan nilaiTerkecil untuk mendapatkan nilai maksimum dan minimum. Hasil akhirnya ditampilkan ke layar menggunakan fmt.Printf, yaitu nilai terkecil ditemukan pada indeks tertentu dan nilai terbesar ditemukan pada indeks tertentu.

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 9 - Modul 10\Guided\Guided-1\minmaxarray.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> 
```

2. minmaxstruct.go

```

1  package main
2
3  import "fmt"
4
5  type mahasiswa struct {
6      nama string
7      nim  string
8      IPK  float64
9  }
10
11 type arrMahasiswa [3]mahasiswa
12
13 func IPKTerdasar(array []mahasiswa) (mahasiswa, int) {
14     var mhsTerdasar mahasiswa = array[0]
15     var idxDitemukan int = 0
16     var i int
17
18     for i = 1; i < len(array); i++ {
19         if array[i].IPK > mhsTerdasar.IPK {
20             mhsTerdasar = array[i]
21             idxDitemukan = i
22         }
23     }
24
25     return mhsTerdasar, idxDitemukan
26 }
27
28 func IPKTerdakil(array []mahasiswa) (mahasiswa, int) {
29     var mhsTerdakil mahasiswa = array[0]
30     var idxDitemukan int = 0
31     var i int
32
33     for i = 1; i < len(array); i++ {
34         if array[i].IPK < mhsTerdakil.IPK {
35             mhsTerdakil = array[i]
36             idxDitemukan = i
37         }
38     }
39     return mhsTerdakil, idxDitemukan
40 }
41
42 func main() {
43     var arrMhs arrMahasiswa
44     var i, j int
45     for i = 0; i < len(arrMhs); i++ {
46         fmt.Println("DATA MAHASISWA INDEKS KE-", i)
47         fmt.Print("Masukkan nama : ")
48         fmt.Scan(&arrMhs[i].nama)
49         fmt.Print("Masukkan NIM :")
50         fmt.Scan(&arrMhs[i].nim)
51         fmt.Print("Masukkan IPK : ")
52         fmt.Scan(&arrMhs[i].IPK)
53         fmt.Println("=====")
54     }
55
56     fmt.Println()
57     for j = 0; j < len(arrMhs); j++ {
58         fmt.Println("DATA MAHASISWA INDEKS KE-", j)
59         fmt.Println("Nama : ", arrMhs[j].nama)
60         fmt.Println("NIM : ", arrMhs[j].nim)
61         fmt.Println("IPK : ", arrMhs[j].IPK)
62         fmt.Println("=====")
63     }
64     fmt.Println()
65
66     var mhsMaks, mhsMin mahasiswa
67     var idxMaks, idxMin int
68
69     mhsMaks, idxMaks = IPKTerdasar(arrMhs[:])
70     mhsMin, idxMin = IPKTerdakil(arrMhs[:])
71
72     fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
73     fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim, idxMaks)
74     fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim, idxMin)
75 }

```

Penjelasan :

Program tersebut digunakan untuk mengelola data mahasiswa berdasarkan nilai IPK. Pertama, program membuat struct mahasiswa yang berisi tiga data, yaitu nama, nim, dan IPK. Kemudian dibuat tipe array arrMahasiswa dengan kapasitas tiga data mahasiswa. Program memiliki dua fungsi utama, yaitu IPKTerbesar untuk mencari mahasiswa dengan IPK tertinggi dan IPKTerkecil untuk mencari mahasiswa dengan IPK terendah. Kedua fungsi tersebut membandingkan nilai IPK setiap mahasiswa dalam array, lalu mengembalikan data mahasiswa beserta indeks tempat data tersebut ditemukan. Pada fungsi main, pengguna diminta memasukkan data tiga mahasiswa berupa nama, NIM, dan IPK. Setelah semua data dimasukkan, program menampilkan kembali seluruh data mahasiswa, kemudian mencari dan menampilkan mahasiswa yang memiliki IPK terbesar dan IPK terkecil beserta nama, NIM, dan indeksinya.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 9 - Modul 10\Guided\Guided-2\minmaxstruct.go"
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : raka
Masukkan NIM : 109082500201
Masukkan IPK : 3.99
=====
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : dharent
Masukkan NIM : 109082500202
Masukkan IPK : 3.90
=====
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : sigit
Masukkan NIM : 109082500203
Masukkan IPK : 3.80
=====

DATA MAHASISWA INDEKS KE- 0
Nama : raka
NIM : 109082500201
IPK : 3.99
=====
DATA MAHASISWA INDEKS KE- 1
Nama : dharent
NIM : 109082500202
IPK : 3.9
=====
DATA MAHASISWA INDEKS KE- 2
Nama : sigit
NIM : 109082500203
IPK : 3.8
=====

=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.99, atas nama raka dengan NIM 109082500201, ditemukan pada indeks ke-0IPK Terkecil : 3.80, atas nama sigit dengan NIM 109082500203, ditemukan pada indeks ke-2
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> |
```

C. Unguided

1. Soal Unguided 1

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Masukan terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

Keluaran terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var N int
7     var berat [1000]float64
8
9     fmt.Println("=== Program Pendataan Berat Anak Kelinci ===")
10    fmt.Print("Masukkan jumlah anak kelinci yang akan ditimbang: ")
11    fmt.Scan(&N)
12
13    for i := 0; i < N; i++ {
14        fmt.Printf("Masukkan berat anak kelinci ke-%d: ", i+1)
15        fmt.Scan(&berat[i])
16    }
17
18    terkecil := berat[0]
19    terbesar := berat[0]
20
21    for i := 1; i < N; i++ {
22        if berat[i] < terkecil {
23            terkecil = berat[i]
24        }
25
26        if berat[i] > terbesar {
27            terbesar = berat[i]
28        }
29    }
30
31    fmt.Println()
32    fmt.Println("=== Hasil Pendataan Berat Anak Kelinci ===")
33    fmt.Printf("Berat anak kelinci terkecil adalah: %.2f kg\n", terkecil)
34    fmt.Printf("Berat anak kelinci terbesar adalah: %.2f kg\n", terbesar)
35 }
```

Penjelasan :

Program tersebut digunakan untuk mendata berat anak kelinci dan mencari berat terkecil serta terbesar. Pada awal program, dibuat variabel N untuk menyimpan jumlah anak kelinci yang akan ditimbang, serta array berat dengan kapasitas 1000 data bertipe float64. Program kemudian menampilkan judul dan meminta pengguna memasukkan jumlah anak kelinci. Setelah itu, pengguna diminta memasukkan berat setiap anak kelinci satu per satu. Data berat yang sudah dimasukkan disimpan ke dalam array. Selanjutnya, program menganggap data pertama sebagai nilai awal untuk berat terkecil dan terbesar. Program kemudian membandingkan setiap data berat yang ada di dalam array untuk menentukan berat paling kecil dan paling besar. Setelah proses perbandingan selesai, program menampilkan hasil pendataan berupa berat anak kelinci terkecil dan berat anak kelinci terbesar dengan format dua angka di belakang koma serta satuan kilogram.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 9 - Modul 10\Unguided\Unguided-no1\beratkelinci.go"
=== Program Pendataan Berat Anak Kelinci ===
Masukkan jumlah anak kelinci yang akan ditimbang: 5
Masukkan berat anak kelinci ke-1: 4.5
Masukkan berat anak kelinci ke-2: 5
Masukkan berat anak kelinci ke-3: 5.5
Masukkan berat anak kelinci ke-4: 4.2
Masukkan berat anak kelinci ke-5: 3.5

=== Hasil Pendataan Berat Anak Kelinci ===
Berat anak kelinci terkecil adalah: 3.50 kg
Berat anak kelinci terbesar adalah: 5.50 kg
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> |
```

2. Soal Unguided 2

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Masukan terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukkan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

Keluaran terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukkan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x, y int
7     var berat [1000]float64
8
9     fmt.Println("=== Program Pendataan Berat Ikan ===")
10    fmt.Print("Masukkan jumlah ikan dan kapasitas tiap wadah: ")
11    fmt.Scan(&x, &y)
12
13    for i := 0; i < x; i++ {
14        fmt.Printf("Masukkan berat ikan ke-%d: ", i+1)
15        fmt.Scan(&berat[i])
16    }
17
18    totalSemua := 0.0
19    jumlahWadah := 0
20
21    fmt.Println("\n=== Total Berat Ikan Setiap Wadah ===")
22    for i := 0; i < x; i += y {
23        totalWadah := 0.0
24
25        for j := i; j < i+y && j < x; j++ {
26            totalWadah += berat[j]
27        }
28
29        jumlahWadah++
30        totalSemua += totalWadah
31        fmt.Printf("Wadah ke-%d: %.2f kg\n", jumlahWadah, totalWadah)
32    }
33
34    rataRata := totalSemua / float64(jumlahWadah)
35
36    fmt.Println("\n=== Hasil Akhir ===")
37    fmt.Printf("Rata-rata berat ikan setiap wadah: %.2f kg\n", rataRata)
38 }
```

Penjelasan :

Program tersebut digunakan untuk menghitung total berat ikan pada setiap wadah dan rata-rata berat ikan per wadah. Pada awal program, pengguna diminta memasukkan jumlah ikan yang akan dijual dan kapasitas ikan dalam setiap wadah. Setelah itu, program meminta pengguna memasukkan berat masing-masing ikan satu per satu dan menyimpannya ke dalam array berat. Selanjutnya, program mengelompokkan data berat ikan berdasarkan kapasitas wadah yang sudah ditentukan. Setiap kelompok ikan akan dijumlahkan untuk mendapatkan total berat pada masing-masing wadah. Total berat setiap wadah kemudian ditampilkan ke layar dengan format dua angka di belakang koma. Selain itu, program juga menjumlahkan seluruh total berat wadah dan menghitung banyaknya wadah yang digunakan. Pada bagian akhir, program menghitung rata-rata berat ikan setiap wadah dengan cara membagi total seluruh berat ikan dengan jumlah wadah, lalu menampilkan hasil rata-rata tersebut dalam satuan kilogram.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 9 - Modul 10\Unguided\Unguided-no2\tarifikan.go"
=== Program Pendataan Berat Ikan ===
Masukkan jumlah ikan dan kapasitas tiap wadah: 5 3
Masukkan berat ikan ke-1: 2.5
Masukkan berat ikan ke-2: 2.2
Masukkan berat ikan ke-3: 3
Masukkan berat ikan ke-4: 2.9
Masukkan berat ikan ke-5: 3.5

=== Total Berat Ikan Setiap Wadah ===
Wadah ke-1: 7.70 kg
Wadah ke-2: 6.40 kg

=== Hasil Akhir ===
Rata-rata berat ikan setiap wadah: 7.05 kg
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> 
```

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 type arrBalita [100]float64
6
7 func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
8     *bMin = arrBerat[0]
9     *bMax = arrBerat[0]
10
11     for i := 1; i < n; i++ {
12         if arrBerat[i] < *bMin {
13             *bMin = arrBerat[i]
14         }
15
16         if arrBerat[i] > *bMax {
17             *bMax = arrBerat[i]
18         }
19     }
20 }
21
22 func rerata(arrBerat arrBalita, n int) float64 {
23     var total float64
24
25     for i := 0; i < n; i++ {
26         total += arrBerat[i]
27     }
28
29     return total / float64(n)
30 }
31
32 func main() {
33     var data arrBalita
34     var n int
35     var min, max float64
36
37     fmt.Print("Masukan banyak data berat balita : ")
38     fmt.Scan(&n)
39
40     for i := 0; i < n; i++ {
41         fmt.Printf("Masukan berat balita ke-%d: ", i+1)
42         fmt.Scan(&data[i])
43     }
44
45     hitungMinMax(data, n, &min, &max)
46     rata := rerata(data, n)
47
48     fmt.Printf("Berat balita minimum: %.2f kg\n", min)
49     fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
50     fmt.Printf("Rerata berat balita: %.2f kg\n", rata)
51 }
```

Penjelasan :

Program tersebut digunakan untuk mencatat dan mengolah data berat balita. Pada awal program, dibuat tipe data `arrBalita` berupa array dengan kapasitas 100 data bertipe `float64`. Program memiliki dua subprogram, yaitu `hitungMinMax` dan `rerata`. Fungsi `hitungMinMax` digunakan untuk mencari berat balita minimum dan maksimum dari data yang sudah dimasukkan, dengan menggunakan pointer `bMin` dan `bMax` agar nilai minimum dan maksimum dapat dikembalikan ke program utama. Sementara itu, fungsi `rerata` digunakan untuk menghitung rata-rata berat balita dengan menjumlahkan seluruh data berat, kemudian membaginya dengan banyaknya data. Pada fungsi `main`, pengguna diminta memasukkan jumlah data berat balita, lalu memasukkan berat balita satu per satu. Setelah semua data tersimpan dalam array, program memanggil fungsi `hitungMinMax` untuk mencari berat terkecil dan terbesar, serta fungsi `rerata` untuk menghitung rata-ratanya. Hasil akhirnya ditampilkan berupa berat balita minimum, berat balita maksimum, dan rerata berat balita dalam satuan kilogram dengan format dua angka di belakang koma.

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> go run "c:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora\Week 9 - Modul 10\Unguided\Unguided-no3\beratbalita.go"
Masukan banyak data berat balita : 5
Masukan berat balita ke-1: 10.5
Masukan berat balita ke-2: 12
Masukan berat balita ke-3: 15
Masukan berat balita ke-4: 13.2
Masukan berat balita ke-5: 13
Berat balita minimum: 10.50 kg
Berat balita maksimum: 15.00 kg
Rerata berat balita: 12.74 kg
PS C:\Users\LENOVO\OneDrive\Documents\109082500201_Raka-Yulia-Vegora> |
```

4. Kesimpulan

Pencarian nilai minimum dan maksimum merupakan salah satu konsep dasar dalam algoritma pemrograman yang digunakan untuk menentukan nilai terkecil dan terbesar dari sekumpulan data. Proses pencarian dilakukan dengan membandingkan setiap elemen data secara berurutan, kemudian memperbarui nilai minimum atau maksimum apabila ditemukan nilai yang lebih kecil atau lebih besar.

Pada praktikum ini, pencarian nilai minimum dan maksimum diterapkan pada beberapa bentuk data, seperti array bilangan, data mahasiswa menggunakan struct, berat anak kelinci, berat ikan dalam wadah, serta berat balita. Dari beberapa latihan tersebut, dapat dipahami bahwa penggunaan array sangat membantu dalam menyimpan banyak data, sedangkan perulangan dan percabangan digunakan untuk mengolah serta membandingkan data tersebut.

Selain itu, praktikum ini juga menunjukkan bahwa algoritma pencarian nilai min/max dapat diterapkan dalam berbagai kasus nyata, seperti mencari IPK tertinggi dan terendah, berat terkecil dan terbesar, serta menghitung rata-rata data. Dengan demikian, mahasiswa dapat memahami cara kerja algoritma pencarian nilai minimum dan maksimum, menerapkannya dalam bahasa pemrograman Go, serta mengetahui pentingnya penggunaan algoritma yang sederhana, efisien, dan mudah dipahami dalam pengolahan data.